КОМП’ЮТЕРНИЙ ПРАКТИКУМ №2

« Криптоаналіз шифру Віженера »

Виконали: Ромашко Мирослава та
Чупай Яна, група ФБ-41
Перевірила: Селюх Поліна Валентинівна

Мета роботи:
Засвоєння методів частотного криптоаналізу. Здобуття навичок роботи та аналізу
потокових шифрів гамування адитивного типу на прикладі шифру Віженера.

ХІД РОБОТИ

2 варіант

1. Дослідження шифру Віженера

Самостійно підібрати текст російською мовою обсягом 2–3 КБ та підготувати його відповідно до алфавіту, прийнятого в роботі.

Реалізувати програмні засоби, необхідні для виконання завдань практикуму. Зашифрувати один і той самий відкритий текст шифром Віженера ключами довжини r =
2, 3, 4, 5, а також одним ключем довжини 10–20 символів.

Для відкритого тексту та всіх одержаних шифртекстів обчислити індекси відповідності,
порівняти їх значення та пояснити одержані результати.

```
2
3  def vigenere_encrypt(text: str, key: str) -> str:
4      r = len(key)
5      return "".join(ALPHABET[(IDX[ch] + IDX[key[i % r]]) % M] for i, ch in enumerate(text))
6
7
8  def index_of_coincidence(text: str) -> float:
9      n = len(text)
0      counts = Counter(text)
1      return sum(c * (c - 1) for c in counts.values()) / (n * (n - 1))
2
```

Функція vigenere_encrypt реалізує шифрування Віженера за формулою $y_i = (x_i + k_{i \bmod r}) \bmod m$.
Для кожної літери відкритого тексту вона бере її номер, додає номер відповідної літери ключа і береться залишок від ділення на 32, після чого за отриманим номером знаходиться літера шифртексту. Операція за модулем 32 гарантує, що після «я» відлік повертається до «а».

Функція index_of_coincidence обчислює індекс відповідності тексту за формулою $I(Y) = 1/(n(n-1)) \cdot \Sigma\, N_t(N_t - 1)$.

За допомогою Counter підраховується, скільки разів N_t зустрілася кожна літера t, для кожної літери обчислюється добуток $N_t(N_t - 1)$, усі добутки сумуються і результат ділиться на $n(n-1)$, де n є довжиною тексту.

```python
if __name__ == "__main__":
    with open("text_task1.txt", encoding="utf-8", errors="ignore") as f:
        plain = clean_text(f.read())

    keys = ["ум", "лес", "ночь", "книга", "криптография"]

    print(f"Довжина відкритого тексту: {len(plain)} літер\n")
    print(f"{'Текст':28s} {'I':>9s}")
    print("-" * 38)
    print(f"{'Відкритий текст':28s} {index_of_coincidence(plain):9.5f}")
    for key in keys:
        cipher = vigenere_encrypt(plain, key)
        print(f"{'r=' + str(len(key)) + ', ключ «' + key + '»':28s} {index_of_coincidence(cipher):9.5f}")
```

В основній частині програми текст зчитується з файлу й очищується функцією clean_text, після чого один і той самий відкритий текст шифрується п'ятьма ключами різної довжини: r = 2 («ум»), r = 3 («лес»), r = 4 («ночь»), r = 5 («книга») та r = 12 («криптография»). Для відкритого тексту та кожного з отриманих шифртекстів обчислюється індекс відповідності.

Результат:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Довжина відкритого тексту: 2354 літер

Текст                              I
--------------------------------------
Відкритий текст                0.05353
r=2, ключ «ум»                 0.03948
r=3, ключ «лес»                0.03823
r=4, ключ «ночь»               0.03773
r=5, ключ «книга»              0.03676
r=12, ключ «криптография»      0.03280
```

Індекс відповідності відкритого тексту становить 0,05353. Це значно більше за $1/32 \approx 0{,}03125$, яке мав би текст із рівноймовірним розподілом літер, і пояснюється це нерівноймовірністю літер російської мови: літери «о», «е», «а» трапляються значно частіше за «ф», «щ» чи «ъ», тому ймовірність збігу двох випадково вибраних літер висока. Після шифрування Віженера індекс падає, причому тим сильніше, чим довший ключ. Отже, з ростом довжини ключа значення I наближається до 1/32.

Це пояснюється будовою шифру. Кожна літера ключа задає власний зсув, тому літери відкритого тексту, що стоять на різних позиціях за модулем r, шифруються різними шифрами Цезаря. Найчастіші літери мови потрапляють у різні літери шифртексту, частоти змішуються, і розподіл літер у шифртексті стає

рівномірнішим. Що більше різних зсувів задіяно, то сильніше вирівнюється розподіл і то ближче індекс до значення для рівноймовірного алфавіту.

2. Криптоаналіз шифртексту

Для наданого викладачем шифртексту (згідно з номером варіанта) виконати криптоаналіз шифру Віженера.

Довжину ключа визначати статистичними методами, описаними в розділі «Необхідні
теоретичні відомості». При пошуку періоду перевірити можливі значення r щонайменше до 30.

У звіті навести результати, на підставі яких було обрано довжину ключа. Недостатньо
вказати лише знайдене значення r.

Дослідити декілька альтернативних кандидатів на довжину ключа. Обґрунтувати вибір
кандидатів та пояснити отримані результати.

```
20
21
22  def language_index(text: str) -> float:
23      # теоретичне значення MI = sum p_t^2 за частотами великого тексту
24      n = len(text)
25      counts = Counter(text)
26      return sum((c / n) ** 2 for c in counts.values())
27
28
29  def split_blocks(text: str, r: int) -> list:
30      # Y_i = y_i, y_(i+r), y_(i+2r), ...
31      return [text[i::r] for i in range(r)]
```

Функція language_index обчислює теоретичне значення індексу відповідності мови MI = Σ p_t^2, де p_t є частотою літери t у великому тексті. Для цього використовується текст із першого практикуму, оброблений за правилами 32-літерного алфавіту. Отримане значення слугує еталоном: якщо індекс блоків шифртексту близький до нього, період визначено правильно, а якщо до 1/32 ≈ 0,031, то ні.

Функція split_blocks розбиває шифртекст на r блоків за правилом $Y_i = y_i, y_\{i+r\}, y_\{i+2r\}, \ldots$, тобто в i-й блок потрапляють літери з позицій i, i+r, i+2r тощо. Якщо r дорівнює

справжньому періоду, усі літери одного блоку зашифровані однією літерою ключа, тобто звичайним шифром Цезаря.

```
33
34  def average_block_index(text: str, r: int) -> float:
35      # Метод 1: середнє значення індексу відповідності по блоках Y_0 ... Y_(r-1)
36      indices = [index_of_coincidence(block) for block in split_blocks(text, r)]
37      return sum(indices) / len(indices)
38
39
40  def coincidences(text: str, r: int) -> int:
41      # Метод 2: D_r = sum delta(y_i, y_(i+r)) — кількість збігів літер на відстані r
42      return sum(1 for i in range(len(text) - r) if text[i] == text[i + r])
43
```

Функція average_block_index реалізує **перший метод** визначення періоду. Вона розбиває шифртекст на r блоків, обчислює індекс відповідності кожного блоку і повертає середнє арифметичне. Усереднення використано замість аналізу великої сукупності окремих значень. Для правильного r блоки мають статистику мови, і середній індекс наближається до 0,055, а для хибних r блоки складаються з літер, зашифрованих різними зсувами, і середній індекс лежить поблизу 1/32.

Функція coincidences реалізує **другий метод** і обчислює статистику збігів $D_r = \Sigma\, \delta(y_i, y_{i+r})$, де δ є символом Кронекера. Вона перебирає всі пари літер на відстані r і рахує, скільки з них однакові. Для відстаней, кратних періоду, збігів істотно більше, бо обидві літери зашифровані однією літерою ключа і збігаються з імовірністю, як у відкритому тексті. Для порівняння різних r у програмі виводиться також нормована величина $D_r/(n - r)$, оскільки кількість пар залежить від r.

Результат:

```
Довжина шифртексту: 7328 літер
Індекс відповідності всього шифртексту: 0.03486
Теоретичний I мови (частоти з ЛР1):     0.05564
I для рівноймовірного алфавіту (1/m):   0.03125

  r  сер. I блоків     D_r   D_r/(n-r)
-------------------------------------
  2        0.03627     258     0.03522
  3        0.03483     236     0.03222
  4        0.03637     262     0.03577
  5        0.03492     240     0.03277
  6        0.03626     232     0.03169
  7        0.04463     244     0.03333
  8        0.03642     220     0.03005
  9        0.03476     238     0.03252
 10        0.03623     253     0.03457
 11        0.03487     247     0.03376
 12        0.03629     247     0.03376
 13        0.03488     212     0.02898
 14        0.05528     396     0.05414
 15        0.03491     252     0.03446
 16        0.03637     253     0.03460
 17        0.03484     262     0.03584
 18        0.03624     252     0.03447
 19        0.03487     247     0.03379
 20        0.03610     233     0.03188
 21        0.04447     276     0.03777
 22        0.03629     219     0.02998
 23        0.03462     228     0.03121
 24        0.03624     243     0.03327
 25        0.03500     238     0.03259
 26        0.03635     234     0.03205
 27        0.03466     237     0.03246
 28        0.05536     396     0.05425
 29        0.03475     236     0.03233
 30        0.03624     235     0.03220
 31        0.03479     260     0.03563
 32        0.03651     240     0.03289
 33        0.03473     228     0.03125
 34        0.03606     229     0.03140
 35        0.04465     270     0.03702
 36        0.03637     236     0.03236
 37        0.03457     220     0.03017
 38        0.03628     257     0.03525
 39        0.03496     253     0.03471
 40        0.03609     242     0.03321
```

```
Топ-5 за середнім I блоків: [28, 14, 35, 7, 21]
Топ-5 за D_r/(n-r):         [28, 14, 21, 35, 17]

Індекси відповідності окремих блоків для найкращих кандидатів:
r = 28: 0.056 0.053 0.057 0.053 0.052 0.064 0.058 0.050 0.053 0.052 0.054 0.054 0.048 0.056 0.063 0.059 0.053 0.058 0.064 0.055 0.058
 0.055 0.055 0.055 0.054 0.055 0.057 0.051
r = 14: 0.060 0.056 0.055 0.056 0.057 0.059 0.057 0.053 0.054 0.054 0.054 0.054 0.052 0.054
r = 35: 0.043 0.045 0.039 0.044 0.047 0.038 0.049 0.040 0.045 0.046 0.049 0.047 0.047 0.046 0.041 0.045 0.045 0.048 0.044 0.044 0.048
 0.038 0.050 0.038 0.047 0.044 0.042 0.049 0.039 0.048 0.046 0.048 0.045 0.045 0.044
```

Довжину ключа визначали двома методами для всіх r від 2 до 40. Довжина шифртексту n = 7328 літер, індекс відповідності всього шифртексту 0,03486 (близький до 1/32 ≈ 0,03125), теоретичний індекс мови 0,05564.

Для **r = 14** середній індекс блоків 0,05528 майже збігається з індексом мови, а для решти значень, не пов'язаних з 14, лежить біля 0,035, тобто на рівні випадкового тексту. Це не наслідок усереднення: усі 14 блоків окремо мають індекси від 0,052 до 0,060, отже кожен блок поводиться як текст, зашифрований шифром Цезаря. Статистика D_r дає ту саму картину: на відстані 14 збігається 5,4% пар літер, що відповідає частоті збігів у мові, тоді як на інших відстанях лише 2,9–3,6%. Тому довжину ключа визначено як **r = 14**.

Кандидатів обирали за двома ознаками: вони потрапили до топ-5 за одним із методів або є дільниками чи кратними знайденого значення.

r = 28. Показники такі самі, як у r = 14, оскільки 28 кратне 14: літери на відстані 28 зашифровані тією самою літерою ключа. Нової інформації це не дає, а блоки вдвічі

коротші, тому їхні індекси розкидані ширше. За принципом найменшого періоду обираємо 14.
r = 7. Дільник 14, показники проміжні: середній індекс 0,04463, D_r/(n−r) = 0,03333. У кожному блоці зібрані літери, зашифровані двома різними літерами ключа, тобто суміш двох шифрів Цезаря. Індекс такої суміші приблизно дорівнює півсумі індексу мови та 1/32, а саме (0,055 + 0,031)/2 ≈ 0,043, що збігається зі спостережуваним значенням. Якби справжній період дорівнював 7, індекс блоків був би близьким до 0,055, тому 7 відхиляємо.
r = 21 і 35. Кратні 7, але не 14. Їхні середні індекси такі самі, як для r = 7, і пояснюються так само: їхній спільний дільник з 14 дорівнює 7, тому блоки також складаються з двох змішаних шифрів Цезаря. Окремі блоки для r = 35 мають індекси 0,038–0,050, помітно нижчі за 0,055.
r = 17. Потрапило до топ-5 за статистикою збігів, але лише через випадкове коливання: D_r/(n−r) = 0,03584 набагато менше за 0,054 для r = 14, а середній індекс блоків 0,03484 лежить на рівні фону. Відхиляємо.

Для решти значень середній індекс блоків лежить у межах 0,0346–0,0365, тобто такий самий, як для випадкового тексту, і періодом вони бути не можуть.

3. Відновлення ключа та відкритого тексту

Шифртекст варіанта 2 містить 7328 літер, алфавіт складається з 32 літер (без «ё», пробілів і розділових знаків);
Довжина ключа r = 14 визначена в завданні 2;
шифртекст розбито на 14 блоків Y_0 … Y_13, де Y_j = y_j, y_{j+14}, y_{j+28}, …;
Блоки Y_0 … Y_5 мають по 524 літери, блоки Y_6 … Y_13 по 523 (бо 7328 = 14·523 + 6).
Кожен блок Y_j зашифровано шифром Цезаря з ключем k_j, тож розподіл літер у блоці є розподілом мови, зсунутим на k_j;
Спосіб 1 (з методички): k_j = (y* − x*) mod 32, де y* це найчастіша літера блоку, а x* = «о» найімовірніша літера мови;
Спосіб 2: вибирається k, що максимізує M_k(Y_j) = Σ_t p_t · f_{(t+k) mod 32}(Y_j). Тут p_t це ймовірність літери t у мові, а f це відносна частота літери в блоці; спосіб 2 *враховує весь розподіл, а не одну літеру, тому він надійніший за спосіб 1.*

```python
    # Спосіб з методички: k = (y* - x*) mod m, x* = 'о'
    ystar = cnt.most_common(1)[0][0]
    k_simple = (ystar - IDX['о']) % M

    # Статистика M_k = sum_t p_t * f_{(t+k) mod m}(Y_j)
    scores = sorted(
        ((sum(P[t] * cnt.get((t + k) % M, 0) / n for t in range(M)), k)
         for k in range(M)), reverse=True)[:3]

    print(f"Y_{j}: n={n}, I={ic(block):.4f}, "
          f"найчастіша літера ШТ '{ALPHABET[ystar]}' -> k='{ALPHABET[k_simple]}' | "
          f"M_k: " + ", ".join(f"'{ALPHABET[k]}'={v:.4f}" for v, k in scores))
    key.append(scores[0][1])

if args.key:
    key = [IDX[c] for c in args.key.lower()]
    print("\nВикористано ключ, заданий вручну.")
```

```
Y_0: n=524, I=0.0598, найчастіша літера ШТ 'ф' -> k='ж' | M_k: 'п'=0.0574, 'м'=0.0441, 'т'=0.0403
Y_1: n=524, I=0.0558, найчастіша літера ШТ 'ь' -> k='о' | M_k: 'о'=0.0560, 'с'=0.0407, 'л'=0.0391
Y_2: n=524, I=0.0552, найчастіша літера ШТ 'я' -> k='с' | M_k: 'с'=0.0558, 'ф'=0.0407, 'о'=0.0400
Y_3: n=524, I=0.0559, найчастіша літера ШТ 'р' -> k='в' | M_k: 'л'=0.0555, 'о'=0.0395, 'и'=0.0395
Y_4: n=524, I=0.0569, найчастіша літера ШТ 'у' -> k='е' | M_k: 'е'=0.0564, 'и'=0.0414, 'в'=0.0409
Y_5: n=524, I=0.0594, найчастіша літера ШТ 'й' -> k='ы' | M_k: 'д'=0.0570, 'б'=0.0434, 'з'=0.0385
Y_6: n=523, I=0.0570, найчастіша літера ШТ 'т' -> k='д' | M_k: 'н'=0.0556, 'р'=0.0425, 'к'=0.0416
Y_7: n=523, I=0.0527, найчастіша літера ШТ 'ц' -> k='и' | M_k: 'и'=0.0544, 'л'=0.0387, 'е'=0.0378
Y_8: n=523, I=0.0544, найчастіша літера ШТ 'о' -> k='а' | M_k: 'й'=0.0551, 'ж'=0.0429, 'м'=0.0381
Y_9: n=523, I=0.0538, найчастіша літера ШТ 'т' -> k='д' | M_k: 'д'=0.0550, 'з'=0.0394, 'б'=0.0379
Y_10: n=523, I=0.0535, найчастіша літера ШТ 'ь' -> k='о' | M_k: 'о'=0.0550, 'л'=0.0410, 'с'=0.0396
Y_11: n=523, I=0.0540, найчастіша літера ШТ 'х' -> k='з' | M_k: 'з'=0.0552, 'к'=0.0415, 'д'=0.0384
Y_12: n=523, I=0.0519, найчастіша літера ШТ 'ь' -> k='о' | M_k: 'о'=0.0538, 'л'=0.0412, 'с'=0.0395
Y_13: n=523, I=0.0536, найчастіша літера ШТ 'ю' -> k='р' | M_k: 'р'=0.0550, 'у'=0.0403, 'н'=0.0390
```

Кожен рядок відповідає одному блоку Y_j, який зашифровано шифром Цезаря з ключем k_j. У рядку наведено: n — довжину блоку; I — індекс відповідності блоку; найчастішу літеру шифртексту y* та ключ k, обчислений за методичкою як (y* − «о») mod 32; три найкращі кандидати на ключ за статистикою M_k з їхніми значеннями.
Індекси блоків лежать у межах від 0,0519 до 0,0598 (середнє 0,0553), тобто кожен блок поводиться як текст, зашифрований шифром Цезаря

У кожному блоці є чітко виражений лідер: найкраще значення M_k лежить у межах 0.0538–0.0574, друге місце 0.0387–0.0441. Розрив між першим і другим кандидатом становить від 0.012 (блок 8) до 0.016 (блок 3). Значення лідерів також близькі до 0.055, і так має бути: при правильному ключі $M_k \approx \Sigma\, p_t^2$, тобто саме до індексу відповідності мови. Тому вибір ключа однозначний.

Порівняння двох способів. Спосіб за методичкою збігся зі способом M_k у 9 блоках із 14 (1, 2, 4, 7, 9, 10, 11, 12, 13). У блоках 0, 3, 5, 6, 8 він дав хибні значення (ж, в, ы, д, а замість п, л, д, н, й). Причина: у цих блоках найчастішою літерою шифртексту відповідає літера «е», а не «о». Різниця між y* і правильним k у всіх п'яти випадках дорівнює 5, а це номер літери «е». На вибірці ≈ 520 літер частоти «о» (≈ 11%) та «е» (≈ 8,5%) досить близькі, тому випадкові коливання могли змінити їхній порядок. Спосіб M_k враховує весь розподіл літер і помилки не дав.

Із трьох найкращих кандидатів у кожному блоці береться перший. З одержаних літер складається ключ п о с л е д н и й д о з о р = «последнийдозор». Він є змістовною фразою, що підтверджує правильність результату.

```
Ключ: последнийдозор

Початок відкритого тексту:
какясмогэтосделатьспросилгесерипочемуэтогонесмогсделатьтымыстоялипосредибескрайнейсеройравнинывзгляднефик
сировалияркихкрасоквцелойкартиненостоиловсмотретьсявотдельнуюпесчинкуитавспыхивалазолотомбагрянцемлазурьюз
еленьюнадголовойзастылобелоесрозовымбудтомолочнуюрекуперемешалискисельнымиберегамидаивыплеснулившебесааеш
едулветерибылохолодномневсегдахолодноначетвертомслоесумраканоэтоиндивидуальнаяреакция
```

Правильність ключа підтверджується трьома ознаками. По-перше, розшифрований текст є зв'язним російським текстом. По-друге, ключ «последнийдозор» є змістовною фразою, що відповідає вказівці методички. По-третє, індекс відповідності розшифрованого тексту дорівнює 0,05544, що збігається з індексом мови (0,0556) і набагато перевищує індекс шифртексту (0,0349).

4. Дослідження впливу помилки у ключі

Один символ правильного ключа змінюється, шифртекст розшифровується повторно. Нехай у ключі змінено літеру `k_j`. Тоді для позицій $i \equiv j \pmod{14}$ буде $x'_i = y_i - k'_j = x_i - (k'_j - k_j) \pmod{32}$, тобто ці літери зсунуться на однакову величину. Решта позицій використовують незмінені літери ключа і розшифруються правильно. Отже, спотворено буде близько 1/14 тексту (≈523 літери), а решта залишиться читабельною. Помилки розташуються періодично, з кроком 14
Змінено літеру на позиції 3 (нумерація з нуля): л → м, зсув δ = +1. Помилковий ключ: «посмеднийдозор

```python
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--file", default="cipher.txt")
    ap.add_argument("--key", default="последнийдозор", help="правильний ключ")
    ap.add_argument("--pos", type=int, default=3, help="позиція зміненої літери (з 0)")
    ap.add_argument("--delta", type=int, default=1, help="зсув літери ключа (mod 32)")
    args = ap.parse_args()

    y = load(args.file)
    key = [IDX[c] for c in args.key.lower()]
    r, n = len(key), len(y)
    bad = key[:]
    bad[args.pos] = (bad[args.pos] + args.delta) % M

    print("--- ПРОГНОЗ ---")
    print(f"Змінюється лише k_{args.pos}. Помилково розшифруються тільки позиції "
          f"i ≡ {args.pos} (mod {r}), тобто ≈ 1/{r} тексту (~{n // r} літер), "
          f"усі зі зсувом -({args.delta}) mod {M}. Решта тексту залишиться правильною.\n")

    ok, ko = decrypt(y, key), decrypt(y, bad)
```

```
--- РЕЗУЛЬТАТ ---
Правильний ключ: последнийдозор
Помилковий ключ: посмеднийдозор
Відрізняється позицій: 524 з 7328 (7.15%), очікувалось ≈ 7.14%
Усі відмінності на i ≡ 3 (mod 14): True
Зсув літер (mod 32): {31}  (очікувалось {31})
```

Параметр	Прогноз	Результат
Кількість змінених літер	≈ 523 (1/14)	524 з 7328 (7,15%)
Позиції змін	i ≡ 3 (mod 14)	усі на i ≡ 3 (mod 14)
Зсув літер	−1 (mod 32)	{31}, тобто −1

Невелика різниця (524 проти 523) пояснюється тим, що 7328 не ділиться на 14 націло, і позиція 3 належить до перших шести залишків, які трапляються на один раз більше.
Значення 31 дорівнює −1 за модулем 32, тобто кожна спотворена літера зсунута на одну позицію назад в алфавіті: я→ю, ь→ы, п→о
Зміни поодинокі, рівномірні й періодичні. Спотворена кожна 14-та літера, що дає плями на місці правильних літер. Смисл тексту легко відновлюється, і цим помилки легко знайти.

```
Правильно:
какясмогэтосделатьспросилгесерипочемуэтогонесмогсделатьтымыстоялипосредибескрайнейсеройравнинывзгляднефик
сировалыркихкрасоквцелойкартиненост
З помилкою:
какюсмогэтосделатыспросилгесериоочемуэтогонеслогсделатьтымыртоялипосредибдскрайнейсеройпавнинывзгляндфик
сировалыркзхкрасоквцелоййартиненост
```

Помилка в одній літері ключа не поширюється на інші позиції, тобто лавинного ефекту немає: шифр Віженера діє як 14 незалежних шифрів Цезаря. Це дозволяє відновлювати ключ по літері окремо, а помилки в окремих блоках не заважають іншим. Крім того, за позиціями спотворень видно, яку саме літеру ключа треба виправити.

Питання для аналізу результатів:

1. Як змінювався індекс відповідності шифртексту при використанні ключів різної довжини? Поясніть отриманий результат.

Для відкритого тексту I = 0,05353, що значно більше за 1/32 ≈ 0,03125 через нерівноймовірність літер мови. Після шифрування індекс падає і зменшується з ростом довжини ключа: 0,03948 (r = 2), 0,03823 (r = 3), 0,03773 (r = 4), 0,03676 (r = 5), 0,03280 (r = 12), тобто наближається до значення для рівноймовірного алфавіту. Кожна літера ключа задає власний зсув, тому літери відкритого тексту шифруються різними шифрами Цезаря, частоти змішуються і розподіл вирівнюється. Що довший ключ, то більше різних

зсувів задіяно і то сильніше вирівнювання. Шифртекст варіанта 2 ($r = 14$) має $I = 0{,}03486$, тобто теж близький до $1/32$.

2. За якими ознаками було визначено найбільш імовірну довжину невідомого ключа?

За двома методами для r від 2 до 40. Для $r = 14$ середній індекс відповідності блоків дорівнює 0,05528, що майже збігається з індексом мови (0,05564), а для решти не пов'язаних з 14 значень лежить у межах 0,0346–0,0365, тобто на рівні випадкового тексту. Усі 14 блоків окремо мають індекси від 0,052 до 0,060, отже високе значення не є наслідком усереднення. Статистика збігів дає ту саму картину: на відстані 14 збігається 5,4% пар літер ($D_{14} = 396$), на інших відстанях лише 2,9–3,6%. Обрано найменше r із високими значеннями за обома методами.

3. Які альтернативні кандидати на довжину ключа ви досліджували, чому обрали саме їх та які результати отримали?

Досліджували значення з топ-5 за будь-яким методом (28, 14, 35, 7, 21, 17) та дільники й кратні знайденого r. Результати такі:

- **$r = 28$** кратне 14: показники такі самі (0,05536 і 0,05425), але блоки вдвічі коротші і їхні індекси розкидані ширше (0,048–0,064 проти 0,052–0,060). Нової інформації не дає, тому за принципом найменшого періоду обрано 14.
- **$r = 7$** дільник 14: середній індекс 0,04463, $D_r/(n-r) = 0{,}03333$. Кожен блок є сумішшю двох шифрів Цезаря, тому $I \approx (0{,}055 + 0{,}031)/2 \approx 0{,}043$, що збігається зі спостережуваним. Відхилено.
- **$r = 21$ і 35** кратні 7, але не 14: індекси (0,04447 і 0,04465) такі самі, як для $r = 7$, бо блоки також складаються з двох змішаних шифрів Цезаря. Відхилено.
- **$r = 17$** потрапило до топ-5 за статистикою збігів лише через випадкове коливання (0,03584 проти 0,054 для $r = 14$), а середній індекс блоків 0,03484 лежить на рівні фону. Відхилено.

4. Яка статистична інформація використовувалася для відновлення окремих символів ключа?

Для кожного з 14 блоків Y_i (приблизно 523 літери) використовували розподіл літер у блоці, бо кожен блок є шифром Цезаря з ключем k_i і його розподіл дорівнює розподілу мови, зсунутому на k_i. За методичкою ключ шукали як $(y^* - \text{«о»}) \bmod 32$, де y^* є найчастішою літерою блоку. Надійніший спосіб максимізує статистику $M_k = \Sigma\, p_i \cdot f_{(t+k) \bmod 32}(Y_i)$, яка враховує весь розподіл літер, а не одну літеру. У кожному блоці лідер мав M_k у межах 0,0538–0,0574, друге місце 0,0387–0,0441, тому вибір однозначний.

5. Як можна перевірити правильність відновленого ключа, не знаючи початкового відкритого тексту?

За трьома ознаками. По-перше, розшифрований текст зв'язний і читається як російський. По-друге, ключ «последнийдозор» є змістовною фразою, як і вказує

методичка. По-третє, індекс відповідності розшифрованого тексту (0,05544) збігається з індексом мови (0,0556) і значно перевищує індекс шифртексту (0,0349). Додатково значення M_k лідерів близькі до 0,055 = Σp□², а розрив між першим і другим кандидатом у кожному блоці становить 0,012–0,016.

6. Як змінився результат розшифрування після внесення помилки в один символ ключа? Поясніть характер і періодичність змін.

Змінили літеру на позиції 3. Спотворилися лише позиції $i \equiv 3 \pmod{14}$: 524 літери з 7328 (7,15%) при очікуваних ≈ 7,14%, тобто кожна 14-та літера, і всі вони зсунуті на $-1 \pmod{32}$, наприклад я→ю, ь→ы, п→о. Решта літер розшифрувалась правильно, і текст лишається читабельним. Невелика різниця (524 проти 523) пояснюється тим, що 7328 не ділиться на 14 націло, а позиція 3 належить до перших шести залишків.

7. Які етапи проведеного криптоаналізу найбільше залежать від обсягу доступного шифртексту? Чому?

Визначення довжини ключа і відновлення літер ключа. Для визначення r індекс обчислюється по блоках довжиною n/r, і чим менше літер у блоці, тим більший розкид значень: для r = 14 індекси блоків лежать у межах 0,052–0,060, для r = 28 уже 0,048–0,064, а для великих r блоки стають надто короткими. Для відновлення літери ключа потрібно достатньо літер у блоці, щоб частоти відрізнялись: на вибірці ≈ 520 літер частоти «о» (≈ 11%) та «е» (≈ 8,5%) близькі, і спосіб за методичкою помилився у 5 блоках з 14 (блоки 0, 3, 5, 6, 8), хоча спосіб M_k помилок не дав. Саме розшифрування та експеримент з помилкою від обсягу не залежать.

8. Які результати ваших експериментів найкраще демонструють слабкість шифру Віженера з періодичним ключем?

По-перше, чіткий пік на r = 14 і кратних значеннях за обома методами: періодичний ключ лишає в шифртексті статистичний слід, який визначає довжину ключа. По-друге, кожен із 14 блоків поводиться як шифр Цезаря, тому шифр зводиться до 14 незалежних і легко зламних шифрів Цезаря, а весь ключ «последнийдозор» відновлено лише за частотами літер. По-третє, помилка в одній літері ключа зачепила лише 1/14 тексту і не поширилась далі, тобто лавинного ефекту немає, і цим шифр відрізняється від сучасних. Змістовний ключ додатково полегшив перевірку.

Висновки:

У роботі досліджено шифр Віженера та проведено частотний криптоаналіз шифртексту варіанта 2. Для відкритого тексту (2354 літери) індекс відповідності становить 0,05353, а після шифрування він зменшується зі зростанням довжини ключа: від 0,03948 для r = 2 до 0,03280 для r = 12, наближаючись до $1/32 \approx 0{,}03125$. Це відбувається тому, що різні літери ключа задають різні зсуви, і розподіл літер у шифртексті вирівнюється.

Довжину ключа невідомого шифртексту (7328 літер) визначено двома методами для r від 2 до 40. Лише для r = 14 і кратного йому r = 28 середній індекс відповідності блоків (0,0553) збігається з індексом мови (0,0556), а частка збігів літер на відстані 14 становить 5,4% проти 2,9–3,6% для інших відстаней. Альтернативні кандидати (7, 21, 35, 17) відхилено, бо вони мають проміжні значення, пояснені спільними дільниками з 14, або є випадковими коливаннями. Для кожного з 14 блоків ключ знайдено за статистикою M_k, яка враховує весь розподіл літер у блоці, і вона виявилася надійнішою за спосіб методички з однією літерою «о»: той помилився у 5 блоках з 14. Отримано змістовний ключ «последнийдозор», а індекс відповідності розшифрованого тексту (0,05544) збігається з індексом мови.

Експеримент із помилкою в одній літері ключа показав, що спотворюється лише кожна 14-та літера (524 з 7328), тоді як решта тексту залишається читабельною. Усі результати демонструють слабкість шифру Віженера з періодичним ключем: він зводиться до r незалежних шифрів Цезаря, період виявляється за статистикою літер, а ключ відновлюється блок за блоком без перебору. Точність криптоаналізу залежить від довжини шифртексту: чим довший ключ, тим коротші блоки і тим менш надійні частотні оцінки.